

Graphectomy Viewer: A Tool for Process-Centric Analysis of Agentic Software Trajectories

Charlie Jyu[✉]

University of Illinois at
Urbana-Champaign
Champaign, USA
cbjyu2@illinois.edu

Shuyang Liu

University of Illinois at
Urbana-Champaign
Champaign, USA
sl225@illinois.edu

Reyhaneh Jabbarvand

University of Illinois at
Urbana-Champaign
Champaign, USA
reyhaneh@illinois.edu

Abstract

We present GRAPHECTORY VIEWER, a web-based tool for interactive, process-centric analysis of software-agent trajectories. Building on the Graphectomy representation introduced in our previous work, GRAPHECTORY VIEWER transforms heterogeneous raw trajectories into phase-aware graphs that connect low-level execution details with higher-level behavioral structures. The tool supports trajectories from multiple agent frameworks and provides interactive graph construction; node-level inspection of thoughts, actions, and observations; search and filtering over large trajectory collections; and Sankey-style summaries of problem-solving phase transitions. These capabilities enable researchers and practitioners to inspect individual executions, identify recurring behavioral patterns, compare successful and failed runs, and analyze large trajectory corpora beyond final task outcomes. To support reproducibility and further research, we release GRAPHECTORY VIEWER as an open-source artifact together with documentation, precomputed graphs, and the large-scale trajectory corpus.

Code: <https://github.com/Intelligent-CAT-Lab/Graphectomy>

Dataset: <https://doi.org/10.5281/zenodo.17364210>

Live Demo: <https://graphectomy-viewer-demo.vercel.app/>

Screencast: <https://youtu.be/Hc4hnfRkuxc>

CCS Concepts

• **Human-centered computing** → **Visualization systems and tools**; *Information visualization*; • **Software and its engineering** → *Software development techniques*.

Keywords

software engineering agents, agent trajectories, process-centric analysis, trajectory visualization, program comprehension

1 Introduction

Software engineering agents increasingly solve real-world programming tasks, including bug localization, code editing, and test generation. Tools such as SWE-agent[21], OpenHands[15], and mini-SWE-agent[14] demonstrate autonomous issue resolution across complex code bases with limited human intervention. However, as these systems become more capable, their executions become harder to understand.

Most existing inspection tools present trajectories as raw logs or step-by-step transcripts, which are useful for replaying individual runs but provide limited support for higher-level analysis. In practice, researchers and practitioners need to answer questions such as: Did the agent correctly localize the bug before editing? Did it repeatedly explore the same files? Did it effectively validate its patch? How do the strategies of successful and failed runs differ? Answering these questions requires structured views that connect individual actions to higher-level problem-solving phases and support comparison across executions.

Prior work introduced GRAPHECTORY [12], a process-centric representation that maps low-level agent actions into semantic phases and graph structures, enabling analyses beyond outcome-centric evaluation. While effective for large-scale, automated studies, it is primarily used as an offline framework. Applying it requires custom scripts and provides limited support for interactively exploring graph structures.

We introduce GRAPHECTORY VIEWER, an interactive, browser-based tool that operationalizes process-centric trajectory analysis by linking low-level execution evidence to higher-level behavioral structures. Given a raw trajectory, GRAPHECTORY VIEWER normalizes framework-specific actions, assigns context-sensitive phase labels, and constructs an interactive, phase-aware directed graph. Users can inspect the thoughts, actions, and observations associated with each node, navigate repeated action occurrences, and suppress low-signal commands to reduce visual clutter. At the corpus level, a companion Sankey diagram aggregates phase transitions across trajectories, supporting the comparison of problem-solving strategies between runs, models, and outcomes. The tool also provides precomputed graphs and trajectories from the 4,000 attempted runs studied in [12], of which 3,973 produced nonempty trajectories, facilitating replication and further analysis.

ACM Reference Format:

Charlie Jyu, Shuyang Liu, and Reyhaneh Jabbarvand. 2026. Graphectomy Viewer: A Tool for Process-Centric Analysis of Agentic Software Trajectories. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3832783.3834593>



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3834593>

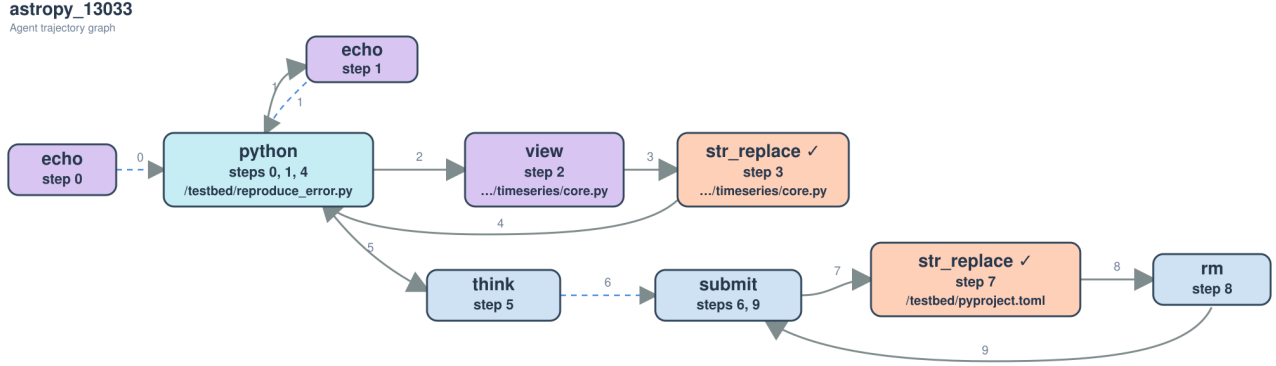


Figure 1: The Graphectory for a SWE-agent trajectory repairing astropy-13033. Phase-colored nodes summarize actions, solid edges show execution flow, and dashed blue edges indicate intra-step command links.

In summary, this paper makes the following contributions:

- **Interactive process-centric trajectory analysis (§3):** We introduce GRAPHECTORY VIEWER, a browser-based tool that connects low-level actions with higher-level behavioral structures through interactive, phase-aware graphs, and aggregate phase-transition summaries.
- **Preliminary user evaluation (§4):** We conduct a within-subject study with five participants comparing GRAPHECTORY VIEWER against the SWE-agent command-line viewer on five trajectory-forensics tasks. GRAPHECTORY VIEWER increased aggregate task accuracy from 8% to 84%, and participants consistently preferred its visual workflow.
- **An open and reusable analysis artifact (§5):** We release the implementation, graph-generation pipeline, documentation, precomputed graph artifacts, and a large-scale trajectory corpus to support reproducible analysis and further research on software-agent behavior.

2 Related Work

Software Agents. The rapid development of agentic software engineering has motivated growing interest in understanding and evaluating agent behavior beyond task-level success [1, 8, 9, 18, 20, 22]. Recent evaluation frameworks examine complementary aspects of agent behavior, including instruction following, alignment with intended goals, and reward design [2, 7, 11, 17, 23]. While these approaches provide metrics and protocols for evaluating agent capabilities, GRAPHECTORY VIEWER supports interactive, process-centric analysis of how software agents navigate, modify, and validate code throughout their execution.

Agent Trajectory Visualization. Existing tools support trajectory inspection and visualization for different analysis and debugging needs. Framework-specific interfaces, such as the SWE-agent Trajectory Inspector and mini-SWE-agent viewer, and tools such as SeaView primarily support replay and inspection of individual executions as formatted text [5, 14, 19]. AgentDiagnose provides general-purpose trajectory diagnostics through execution traces and embedding-based visualizations [16], while AgentLens

and AGDebugger support visual analysis, debugging, and steering of LLM-based agent systems [6, 13]. In contrast, GRAPHECTORY VIEWER maps heterogeneous software-agent trajectories into a graphical process-centric representation. It also supports corpus-level comparison through aggregate phase-transition summaries.

3 Tool Overview

The main browser interface is organized around a navigation sidebar and a main visualization pane. As shown in Figure 2, users begin with the *Data source* panel by supplying the trajectories and report. The sidebar then populates a searchable instance list with status badges and trajectory counts.

For trajectory-level analysis, selecting an instance from the sidebar loads a Graphectory into the main canvas. Figure 1 shows a graphectory of SWE-agent using DeepSeek to solve issue astropy-13033 from SWE-bench Verified. The agent begins (step 0) by creating a script to replicate the issue from the problem description and running it with python. Steps 0-2 across the first four purple nodes represent the localization steps the agent takes to find the source of the issue. The second node (representing steps 0,1,4) is colored both as purple for localization and blue for validation because the command it represents occurs across multiple trajectory steps that belong to different phases. Step 3 shows the agent patching the code after localizing the issue then validating its correctness in step 4 by running the issue replication script. After thinking in step 5 about step 4’s validation results, the model submits, briefly cleaning up the code in steps 7 and 8 before finally submitting. Solid edges show the main execution flow, while dashed blue edges represent intra-step links between multiple commands issued within the same trajectory step. Arrowhead size of an edge shows the length of the thought in the node it points to.

Clicking a node opens the inspection sidebar on the right and the file footprint menu. The sidebar shows the raw step’s thought-action-observation tuple. When a node is referenced by multiple steps, the interface provides tabs for each step. The file footprint appears on the left of the canvas to show a visual representation of the files viewed and edited over the course of the trajectory. It

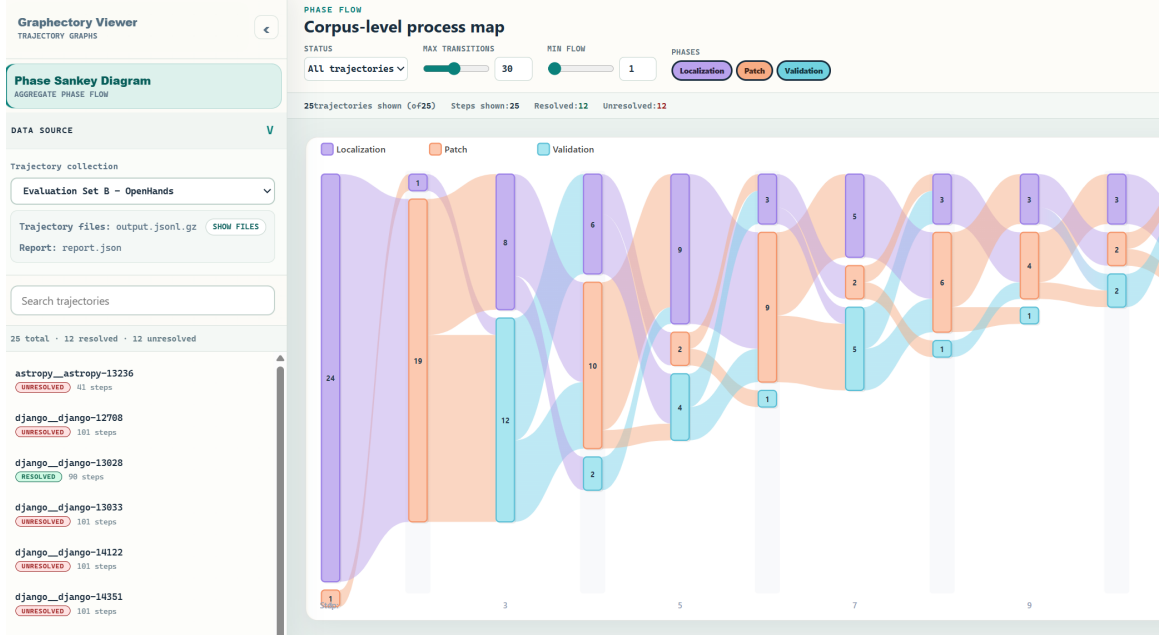


Figure 2: The Phase Sankey Diagram interface with status filtering, phase chips, and sliders for transition depth and minimum displayed flow.

also shows when each file was interacted with, providing a clear file level representation of how the agent acted.

The Phase Sankey Diagram, shown in Figure 2, provides the second major interface mode by summarizing each run as a sequence of meaningful phase transitions. Users can filter the displayed trajectories by status, adjust the maximum number of displayed transitions, suppress low-frequency flows with a minimum-flow slider, and enable or disable entire phase categories.

The backend converts trajectory directories or OpenHands-style output .jsonl files into a shared process-centric graph representation. First, `commandParser.py` uses Bash-aware parsing to normalize shell commands, chained actions, and framework-specific tool calls into structured records [4]. Next, `mapPhase.py` assigns context-sensitive localization, patch, validation, or general labels following the Graphectomy taxonomy [12]; for example, `pytest` may represent localization before an edit and validation afterward. Finally, `buildGraph.py` deduplicates repeated actions into shared nodes while retaining their information in the graph. The same pipeline supports serialized graph export, while the Sankey visualizer collapses repeated neighboring phases into compact paths for corpus-level comparison.

4 User Evaluation

We conducted a preliminary within-subject usability study with five participants whose backgrounds ranged from basic computer science knowledge to professional software development and AI experience. Participants were recruited through convenience sampling and were told that the interfaces, rather than their individual ability, were being evaluated. Less experienced participants received

a brief introduction to software agents, SWE-agent trajectories, and the fields shown by the viewers.

Each participant used both the SWE-agent command-line trajectory viewer and Graphectomy Viewer to complete the same five trajectory-forensics tasks. The tasks required identifying a trajectory containing: (1) a command repeated at least 20 times, (2) thought continuation across consecutive steps, (3) at least two command loops, each repeating the same command and parameters at least three times, (4) a non-submission thought longer than 700 characters, and (5) an observation longer than 20,000 characters.

We created two evaluation sets of 25 trajectories generated using SWE-agent with Mistral Devstral Small on SWE-bench Verified tasks. For each participant, one set was assigned to the command-line viewer and the other to Graphectomy. We alternated the assignment across participants to reduce bias from differences between the datasets. Participants used the command-line viewer first and Graphectomy second, with a 10 minute limit for all five tasks in each condition. We recorded the number of correct answers and the completion time when all tasks were finished before the limit. Participants were then invited to rate both interfaces and provide open-ended feedback.

The command-line viewer yielded 2 correct answers across 25 attempts (8%), and no participant completed all five tasks within 10 minutes. Graphectomy yielded 21 correct answers (84%); three participants completed all five tasks in 5:13.80, 7:10, and 7:58. Participants rated Graphectomy between 8 and 9 out of 10 and consistently preferred its visual presentation. They highlighted the usefulness of phase colors, node and arrow encodings, raw-step inspection, and the Sankey view. Suggested improvements included a clearer

legend, more accurate arrow scaling, and a tutorial; all of which we subsequently added.

These results provide preliminary evidence that Graphectory improves performance on the studied trajectory-forensics tasks. However, the small convenience sample, fixed interface order, and task design specifically targeting queries that may favor visual search limit generalizability of these results.

5 Utility and Quality

To evaluate the tool more systematically, we measured it over the released trajectory corpus used in our broader Graphectory workflow. This corpus contains 3,973 trajectories spanning eight collections (27 of 4,000 runs resulted in empty trajectories): four SWE-agent runs and four OpenHands runs over SWE-bench Verified tasks. Across the full corpus, raw trajectories average 35,664 lines and 3,047,249 characters, but the corresponding Graphectories average only 34.53 nodes and 49.60 edges. Even at this scale, the viewer preserves the underlying evidence because each node remains linked to its full thought, action, and observation tuples.

Table 1 shows that this compression is consistent across agents and models, while also revealing meaningful behavioral differences. For example, SWE-agent with DeepSeek-V3 yields especially compact graphs, averaging only 14.99 nodes and a shortest execution path of 7.50 despite raw trajectories averaging 15,895 lines. At the other extreme, SWE-agent with Claude Sonnet 4 produces much larger logs, averaging 144,587 lines and over 10 million characters, yet the corresponding graphs still average only 46.49 nodes. Deduplication is substantial across all collections, reducing effective graph size by 9.60% to 28.22% on average depending on the source. These reductions quantify how repeated actions are merged into shared nodes, revealing the backtracking and looping behavior that would otherwise be hidden in long, linear trajectory logs.

Beyond compression, the viewer enabled several recurring behavioral patterns to become apparent across large trajectory collections. We frequently observed agents entering repeated localization or validation loops on difficult tasks, repeatedly revisiting the same actions without transitioning into new phases or making meaningful progress. The process-centric representation additionally revealed that thought length alone is an unreliable proxy for reasoning complexity: agents sometimes execute complex multi-stage commands following only shallow thoughts, while lengthy reasoning traces may precede relatively trivial actions. These patterns were difficult to identify consistently from raw transcripts alone, but become substantially more visible through aggregate graph and phase-transition analysis.

6 Conclusion

We presented GRAPHECTORY VIEWER, an interactive tool that transforms heterogeneous software-agent trajectories into phase-aware graphs and aggregate transition summaries. Our evaluation provides preliminary evidence that the viewer can support process diagnosis over raw-log inspection in the studied tasks, while the corpus analysis illustrates how its phase-aware representation can expose agent problem-solving strategies. While the current system relies on domain-specific mapping rules, its adapter-based architecture facilitates extension to new domains and agent frameworks.

Table 1: Trajectory and graph average statistics across released SWE-agent and OpenHands collections.

Source	N	lines	nodes	dedup.	path
All collections	3973	35,664	34.53	14.71%	16.70
OH Claude-4	500	2,004	59.90	9.60%	30.49
OH DeepSeek-V3	500	5,219	25.98	11.87%	13.75
OH DeepSeek-R1	474	3,681	19.46	11.07%	12.57
OH Devstral	500	12,446	72.53	10.37%	35.66
SA DeepSeek-V3	499	15,895	14.99	20.71%	7.50
SA Claude-4	500	144,587	46.49	11.94%	16.35
SA DeepSeek-R1	500	7,466	12.57	13.71%	6.40
SA Devstral	500	92,314	23.50	28.22%	10.66

By releasing the implementation, documentation, and precomputed corpus, we aim to support reproducible analysis and debugging of increasingly complex software-agent executions.

7 Data Availability Statement

The code and viewer are publicly available under the University of Illinois/NCSA Open Source License [10]. The raw trajectory corpus is archived in Zenodo [3] under DOI [10.5281/zenodo.17364210](https://doi.org/10.5281/zenodo.17364210). The repository provides installation instructions, usage examples, sample inputs, graph-export scripts, trajectory-metric scripts, and a screencast. A live demo is available at <https://graphectory-viewer-demo.vercel.app/>.

References

- [1] Mohamad Abou Ali, Fadi Dornaika, and Jinan Charafeddine. 2025. Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions. *Artificial Intelligence Review* 59, 1 (2025), 11. doi:10.1007/s10462-025-11422-4
- [2] Sreemae Akshathala, Bassam Adnan, Mahisha Ramesh, Karthik Vaidhyanathan, Basil Muhammed, and Kannan Parthasarathy. 2025. Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems. arXiv:2512.12791 [cs.MA] doi:10.1145/3786167.3788414
- [3] Anonymous-Dev. 2025. Raw Trajectories for SWE-agent/OpenHands on SWE-Bench Verified for Graphectory reproduction. doi:10.5281/zenodo.17364210
- [4] bashlex developers. 2026. bashlex. <https://github.com/idank/bashlex>. Python parser for Bash. Accessed: 2026-04-28.
- [5] Timothy Bula, Saurabh Pujar, Luca Buratti, Mihaela Bornea, and Avirup Sil. 2025. SeaView: Software Engineering Agent Visual Interface for Enhanced Workflow. arXiv:2504.08696 [cs.SE] doi:10.48550/arXiv.2504.08696
- [6] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. arXiv:2503.02068 [cs.HC] doi:10.1145/3706598.3713581
- [7] Sravani Gunnu, Shanmukha Guttula, and Hima Patel. 2025. CIFE: Code Instruction-Following Evaluation. *arXiv preprint arXiv:2512.17387* (2025). doi:10.48550/arXiv.2512.17387
- [8] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79. doi:10.1145/3695988
- [9] Zhonghao Jiang, David Lo, and Zhongxin Liu. 2025. Agentic Software Issue Resolution with Large Language Models: A Survey. *arXiv preprint arXiv:2512.22256* (2025). doi:10.48550/arXiv.2512.22256
- [10] Charlie Jyu, Shuyang Liu, and Reyhaneh Jabbarvand. 2026. Graphectory. <https://github.com/Intelligent-CAT-Lab/Graphectory>
- [11] Christopher Koch and Joshua A. Wellbrock. 2026. Beyond Task Success: An Evidence-Synthesis Framework for Evaluating, Governing, and Orchestrating Agentic AI. arXiv:2604.19818 [cs.SE] doi:10.48550/arXiv.2604.19818
- [12] Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhaneh Jabbarvand. 2026. Process-Centric Analysis of Agentic Software Systems. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 163 (April 2026), 28 pages. doi:10.1145/3798271

- [13] Jiaying Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyuan Hu, Yuchen Peng, and Wei Chen. 2024. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems. *IEEE Transactions on Visualization and Computer Graphics* (2024). doi:10.1109/TVCG.2024.3394053
- [14] mini-SWE-agent documentation. 2026. Overview. <https://mini-swe-agent.com/latest/>. Accessed: 2026-04-28.
- [15] OpenHands. 2024. OpenHands: Code Less, Make More. <https://github.com/OpenHands-AI/OpenHands>
- [16] Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Suzhou, China, 207–215. doi:10.18653/v1/2025.emnlp-demos.15
- [17] Mohit Raghavendra, Anisha Gunjal, Bing Liu, and Yunzhong He. 2026. Agentic Rubrics as Contextual Verifiers for SWE Agents. *arXiv preprint arXiv:2601.04171* (2026). doi:10.48550/arXiv.2601.04171
- [18] Leon Staufer, Kevin Feng, Kevin Wei, Luke Bailey, Yawen Duan, Mick Yang, A. Pinar Ozisik, Stephen Casper, and Noam Kolt. 2026. The 2025 AI Agent Index: Documenting Technical and Safety Features of Deployed Agentic AI Systems. *arXiv:2602.17753* [cs.CY] doi:10.48550/arXiv.2602.17753
- [19] SWE-agent documentation. 2026. Trajectory Inspector. <https://swe-agent.com/latest/usage/inspector/>. Accessed: 2026-04-28.
- [20] Huaning Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. *arXiv preprint arXiv:2508.11126* (2025). doi:10.48550/arXiv.2508.11126
- [21] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1601
- [22] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on Evaluation of LLM-based Agents. *arXiv:2503.16416* [cs.AI] doi:10.48550/arXiv.2503.16416
- [23] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy K. Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Antony Kellermann, Jasjeet S. Sekhon, Jacob Steinhardt, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. 2025. Establishing Best Practices in Building Rigorous Agentic Benchmarks. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. doi:10.48550/arXiv.2507.02825

Received 2026-05-11; accepted 2026-06-19